

AccessMatrix Common Hardening Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AccessMatrix Common Hardening Guide.....	3
2. Introduction to AccessMatrix Hardening.....	4
3. Securing Bundled Tomcat.....	5
3.1. Tomcat and Operating System Hardening.....	6
3.2. Validate Tomcat Hardening.....	16
4. AccessMatrix Server Hardening.....	37
4.1. API Access.....	38
4.2. Key Management.....	40

1. Preface - AccessMatrix Common Hardening Guide

This guide contains instructions on the hardening of the AccessMatrix system to increase its security.

Intended Reader

It is intended for system administrators to set up, operate and maintain the AccessMatrix system.

Document Scope

This document provides instructions on how to harden various AccessMatrix components.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Release Notes*
- [AccessMatrix Common Administration Guide](#)

2. Introduction to AccessMatrix Hardening

AccessMatrix has a number of components. The hardening process of the components will be divided into several sections in this document.

Component	Internal Dependency	External Dependency
Security Registry	- None -	JDBC-Compliant Database
AccessMatrix (AM) Server	Security Registry	Web App Server
Web Applications (USO SSF, USO SSO etc.)	AccessMatrix (AM) Server	Web App Server
USO Client	USO Agent	- None -



Note: The term <ACMATRIX_ROOT> will be used to refer to the root or installation directory of AccessMatrix on both Windows and Unix platforms.

3. Securing Bundled Tomcat

Tomcat is commonly deployed as part of an all-in-one installation of AccessMatrix. The following sections contain information on how to secure the bundled Tomcat:

- [Tomcat and Operating System Hardening](#)
- [Validate Tomcat Hardening](#)

3.1. Tomcat and Operating System Hardening

This section contains the steps to harden AccessMatrix's bundled Tomcat and the operating system used to run AccessMatrix.

Run as a Specific User and Restrict Permissions

As part of the best security practices for running applications, it is not recommended to run as privileged users (such as root users or Administrators). Running as a specific user dedicated for AccessMatrix also allows the next step to secure the directory access to restrict other users to access or modify AccessMatrix or Tomcat files. Furthermore, in the occasion when the Tomcat server is compromised, then the damage will be limited that it does not affect other directories belonging to other users or system-related configuration.

Linux	As part of deployment, there should be a user (and optionally, a group) dedicated to running AccessMatrix Server. The user creation step is not covered in this guide. For the commands below, set the AM_TOMCAT env variable to full path of <code><ACMATRIX_HOME>/tomcat/</code> for easy reference: <code>AM_TOMCAT=<ACMATRIX_HOME>/tomcat</code> Example: <code>AM_TOMCAT=/opt/am/tomcat</code> The following directories need to be restricted in the following manners • The \$AM_TOMCAT directory and its subdirectories should be set with the <code>o-rwx,g-w</code> permission (removing all permissions from world and removing write permission from group) • Files in \$AM_TOMCAT and its subdirectories should be set with <code>o-rwx</code> permission (removing all permissions from world) • Example: <code>find \$AM_TOMCAT -type d -exec chmod o-rwx,g-w {} +</code> <code>find \$AM_TOMCAT -type f -exec chmod o-rwx {} +</code> • \$AM_TOMCAT/db is used to store the Derby database file. For production environment, Derby should not be used; thus, this directory can be removed. • Secure sensitive configuration files that may contain keystore or database password, encryption utility's secret file, etc. Permission should be set with <code>o-rwx,g-rwx</code> (removing all permissions from world and group) • Example: <code>chmod 600 \$AM_TOMCAT/conf/server.xml</code> <code>\$AM_TOMCAT/conf/Catalina/localhost/am5.xml</code>
-------	---

Windows	It is best to create a Service user specific for running AccessMatrix Server and configure the AccessMatrix Server windows service to run as that user. Creating Service user is not part of this document as it depends on the organization standard. 1. Run the Tomcat service configuration window by running <code><ACMATRIX_ROOT>\tomcat\bin\am5w.exe</code> or by running Monitor AccessMatrix Server and right-clicking on the AccessMatrix monitoring icon in the taskbar and clicking on Configure. 2. Click on the Log On tab. 3. Enter the Service user that will be used to run AccessMatrix Server and its credential. 4. Click OK to save the changes. If you have an issue with Running AccessMatrix Server, you may need to modify Group Policy to allow the Service user to "Logon as a service" (in Computer Configuration - Windows Settings - Security Settings - Local Policies - User Rights Assignment). Set directory and file permissions to the <code><ACMATRIX_ROOT>\tomcat\</code> directory such that: • Remove permissions from other users and groups (you may need to remove inherited permissions in order to do this) • Set Full Control permissions to the Service user for AccessMatrix Server (replace all its subdirectories inherited permissions) • Only provide Full Control permission to Windows Administrator user used to configure AccessMatrix Server • You may provide other user a read access for operational and troubleshooting purposes as needed (e.g. to retrieve log files) • Note that configuration files containing sensitive information, such as keystore or database password, encryption utility's secret file, must not be readable by other users unnecessarily. Remove <code><ACMATRIX_HOME>\tomcat\db\</code> if not used. This directory is used by Derby database, which is not meant for Production use. If it is not used, the directory may be removed.
---------	--

Shutdown Port

Tomcat normally starts with the shutdown port opened. This port is used to trigger the shutdown process when the shutdown/stop command (`catalina stop`) is run. By default, the port is opened with binding to localhost. This means the port is accessible only from that particular server.

To better protect the Tomcat instance, the shutdown port can be disabled by setting the shutdown port to -1 in **server.xml**.

1. Open `<ACMATRIX_ROOT>\tomcat\conf\server.xml`.
2. Modify the port in Server tag to -1:

XML

```
<Serverport="-1"shutdown="SHUTDOWN">
```

Linux only

However, with the shutdown port disabled, CATALINA_PID needs to be set to allow the shutdown sequence to function by sending signal to the Tomcat process. CATALINA_PID can be set in **setenv.sh**.

1. Open <ACMATRIX_ROOT>/tomcat/bin/setenv.sh
2. Add the following line at the end (directory name can be changed as needed as long as the OS user used to run Tomcat has read and write access to the directory):

Bash (Unix Shell)

```
CATALINA_PID="$CATALINA_BASE/work/catalina.pid"
```

Use HTTPS

By default, the bundled Tomcat has two ports to serve requests: 8080 and 8443. The port 8443 has an optional client-authentication SSL (mTLS) enabled, which is meant for API access and it is also used for the web access (such as Admin Console). The port 8080 is a non-SSL port. This needs to be removed.

Port	HTTP/HTTPS	mTLS	Usage	To be removed
8080	HTTP	No	Web (Admin Console, CLP, USO Server)	Yes
8443	HTTPS	Yes (optional)	API Web (Admin Console, CLP, USO Server)	No

By default, the bundled Tomcat is configured using testing SSL certificates and truststores. For production use, you need to generate a proper certificates that are trusted by the applications or browsers that may access SSL ports.

Port Segregation

An optional step to harden AccessMatrix Server further is to segregate the ports used for API and web access. This allows the firewall to limit the access further such that AccessMatrix clients that need access via API can access the port meant for API and to restrict users who need to have web access to access via the web access port.

To have segregation of ports:

-
1. Copy the existing connector configuration in **server.xml**.
 2. Set the port to a new port value.
 3. Configure the SSLHostConfig certificateVerification value ("required" to enforce mTLS for API access). Click [here](#) for more details.
 4. Save the changes.

Encrypt HTTPS Keystore Password

AccessMatrix provides encryptor utility (**encryptor.bat** or **encryptor.sh** in <ACMATRIX_HOME>/bin/) that can be used to encrypt the Keystore password so that it does not appear in plaintext in the **server.xml**. The encryption itself requires a secret to be provided in a separate file. Access to the file needs to be secured using file permission of the OS (Linux: 600, Windows: allow only user used to run AM that may access the file). This encryption helps to avoid accidental exposure when server.xml needs to be provided to other party (e.g. i-Sprint support) for support purpose.

Refer to the [Common Administration Guide - Encryptor Utility](#) section. Ensure you choose AesEncryptor or Aes256Encryptor, and provide the user salt (secret) using file.

Encrypt Database User Password

AccessMatrix Server requires database connection that is defined in <ACMATRIX_HOME>/tomcat/conf/Catalina/localhost/am5.xml. The database password can be encrypted similar to the HTTPS Keystore Password mentioned earlier. Encrypting it helps to avoid accidental exposure when **am5.xml** needs to be provided to other party (e.g. i-Sprint support) for support purpose.

Refer to [Common Deployment Guide - Encryptor Utility](#) section. Ensure you choose AesEncryptor or Aes256Encryptor, and provide the user salt (secret) using file.

Disable Client-facing Stacktrace

By default, Tomcat will display the Java stacktrace when an unhandled exception occurs. The information is useful for troubleshooting purpose and normally AccessMatrix is not deployed as user facing directly. However, to make the deployment more secure, the stacktrace needs to be hidden and you may also choose to display your own error page.

Error page in Tomcat can be set via common **web.xml** (in `<ACMATRIX_ROOT>/tomcat/conf/`) or in the webapp's **web.xml** (`/WEB-INF/web.xml`). It depends whether you choose to apply the same configuration for all web applications or to apply different configuration in each web application.

1. Open `<ACMATRIX_ROOT>/tomcat/conf/web.xml` or web application's `/WEB-INF/web.xml`.
2. Add the following `<error-page>` configuration:

XML

```
<webapp>
  ...

  <error-page>
    <location>/error.htm</location>
  </error-page>
</webapp>
```

You may choose to use a different page. The location specified is relative to the web application. For example, `/error.htm` is referring to the web application's root/base directory (e.g. `<ACMATRIX>tomcat/webapps/am5/error.htm`)

The above options will only take effect only when a web application or context path is being accessed. In order to also handle error that happens before reaching the web application itself, `ErrorReportValve` needs to be configured in **server.xml**.

1. Open `<ACMATRIX_ROOT>/tomcat/conf/server.xml`.
2. Uncomment the following `<Valve>` within the `<Host>` element:

XML

```
<Valve
  className="org.apache.catalina.valves.ErrorReportValve"
  errorCode.0="webapps/errors/error.html"

  exceptionType.java.lang.Throwable="webapps/errors/error.html"
  showReport="false"
  showServerInfo="false" />
```

Error page configuration will follow these precedence (highest priority to lowest precedence):

1. `<error-page>` in the web application's **web.xml**.
 2. `<error-page>` in the common **web.xml**.
 3. `ErrorReportValve` in **server.xml**.
-

AccessMatrix REST API

AccessMatrix REST API returns HTTP 400 when an exception/fault is returned. This will not be affected by the error page. Tomcat will still return the intended response from the API instead of returning an error page.

References

- **ErrorReportValve:** <https://tomcat.apache.org/tomcat-9.0-doc/config/valve.html>
- **<error-page>:** <https://tomcat.apache.org/tomcat-9.0-doc/api/org/apache/tomcat/util/descriptor/web/ErrorHandler.html>

Other Security Settings

Tomcat has many configurations. Some configurations may directly or indirectly cause some security concerns. AccessMatrix's bundled Tomcat has been partially hardened by default, such as removing the unnecessary web applications and setting certain configurations that harden the Tomcat.

The hardening steps above are additional steps that can be done to complete the hardening process. There are other common security hardening steps that are not mentioned above for several reasons:

- These steps are already configured by default
- These steps are not applicable to the AccessMatrix Server
- These steps are unnecessary in the context of the AccessMatrix Server

If you wish to understand more about these settings, you can refer to the following sections for the details:

Security Manager

Security Manager is used to run Tomcat in a sandbox. This can further limit what the application can do, such as preventing access to all files, or preventing a malicious web application to kill the Tomcat Java process. Since the bundled Tomcat is dedicated to running the AccessMatrix server with the additional hardening steps (such as running the process using a dedicated user), running with Security Manager is not required.

Server Information

By default, the AccessMatrix server has been configured to not return server information (such as Tomcat version).

Security Configuration Regarding Tomcat Users

By default, the manager web application that relies on the Tomcat users database is removed. The AccessMatrix server has its own authentication mechanism and does not make use of Tomcat users.

AJP (Apache JServ Protocol)

The AJP connector is disabled by default.

Session Facade Recycling

By default, session facade recycling is disabled for the AccessMatrix server. This means that, instead of reusing facades between requests, a new facade will be created for each request. This mitigates a potential security concern where recycling session facades between requests may result in information leakage.

Additional Path Delimiters

By default, all additional path-delimiters are prohibited. This prevents attackers from exploiting path delimiters to gain access to applications or areas that should not be visible to them.

Effective web.xml Logging

The effective **web.xml** is a combination of a web application's **web.xml** with any defaults configured by Tomcat and any **web-fragment.xml** files and annotations discovered. The effective **web.xml** of each AccessMatrix application is logged (at the INFO level) when it is started.

By default, the scanning of fragments and annotations is disabled. This means that each AccessMatrix application's **web.xml** file is the only metadata considered for the effective **web.xml**. This addresses security concerns that may arise as a result of fragments and annotations being discoverable via the effective **web.xml**.

Servlet Filters

The following sections contain information on security-related servlet filters. These filters are pre-configured to set security-related HTTP response headers that address potential vulnerabilities.

Cache Control Policy

AccessMatrix enforces a set of secure caching policies to prevent unauthorized/unauthenticated users from accessing sensitive information that was previously cached. Without such policies, sensitive information that is cached may be accessible via the browser's cache or by using the browser's **Back** button.

By default, AccessMatrix uses the following HTTP response headers and directives for cache control:

- **Cache-Control**

This header controls caching in browsers and shared caches (e.g. proxies, CDNs). The following directives are configured by default:

- **no-cache** - This setting means that responses can be cached, but browsers must submit a validation request to the origin server before the response can be reused.
- **no-store** - This setting means that browsers cannot cache responses.
- **must-revalidate** - This setting means that a response can only be cached and reused while fresh. After it becomes stale (expires), browsers must revalidate the stored response with the server before it can be re-accessed.
- **max-age=0** - This setting means that the cached copy of a resource expires immediately upon being cached (in other words, a resource is never considered fresh). Used in conjunction with **must-revalidate** to ensure that a new request must always be sent to the server for validation before the resource can be re-accessed.

- **pragma**

This header is deprecated, and is included only for backwards compatibility with HTTP/1.0 clients. The following directives are configured by default:

- **no-cache** - Same as **Cache-Control: no-cache**.

- **Expires**

This header contains the date/time after which the response is considered expired. The following directives are configured by default:

- **0** - This setting means that the resource is immediately considered expired.
-

It is recommended that you do not change these settings.

However, it is possible to do so if necessary. Navigate to `<ACMATRIX>/tomcat/webapps/am5/WEB-INF/`, open **web.xml**, and search for the Cache Control Header section to find the cache control filter and its corresponding settings. The relevant parameters are `cacheControlOptions`, `pragmaOptions`, and `expiresOptions`.

Content Security Policy (CSP)

AccessMatrix enforces a strict content security policy. This implementation only allows hashed scripts or scripts with the correct nonce value generated on the server. For other resources, the policy generally allows only loading them from the current origin (i.e. itself). In instances where this policy cannot be enforced without disabling the functionality of AccessMatrix or its components, the next most secure option is enforced. Controlling the loading of resources can help to prevent cross-site scripting attacks, in which malicious scripts are injected into the web app and executed on the user's browser.

i **Note:** Specifically, current origin refers to the same domain (with the same URL scheme and port number) as the client making the HTTP request.

By default, AccessMatrix uses the Content-Security-Policy HTTP header with the following directives for content security:

- **script-src 'sha256-`<base64-value>`'** - This setting means an inline script can only be loaded and executed if its hash value matches the allowed values in the CSP header. An external file in a `<script>` element can only be loaded and executed if all the valid hash values in its integrity attribute match the allowed values in the CSP header. The **Subresource Integrity** feature additionally checks that the downloaded file has the indicated hash value, and therefore has not been modified.

i **Notes:** Refer to [this page](#) for information on **Subresource Integrity**.

- **script-src 'strict-dynamic'** - The setting means that the trust explicitly given to a script present in the markup, by accompanying it with a nonce or a hash, shall be propagated to all the scripts loaded by that root script. At the same time, any allowlist or source expressions such as 'self' or 'unsafe-inline' are ignored.
 - **script-src 'unsafe-eval'** - This setting means that the use of `eval()`, as well as other unsafe methods for creating code from strings for JavaScript, is allowed. This includes not only URLs
-

loaded directly into <script> elements, but also things like inline script event handlers (onclick) and XSLT stylesheets that can trigger script execution.

- **default-src 'self'** - This setting means that only resources from the current origin can be loaded. This directive serves as a fallback for various other CSP fetch directives; if these fetch directives are absent, the value specified for **default-src** is used for them.



Notes: Refer to [this page](#) for information on which other CSP fetch directives are covered by **default-src**.

- **style-src 'self'** - This setting means that only stylesheets from the current origin can be loaded.
- **style-src-attr 'unsafe-inline'** - This setting means that inline styles defined using HTML attributes are allowed to be executed.
- **style-src-elem 'self' 'unsafe-inline'** - This setting means that inline styles from the same origin, those contained within <style> elements, and <link> elements with the rel="stylesheet" attribute, are allowed to be executed.
- **img-src 'self'** - This setting means that only images and favicons from the same origin can be loaded.
- **frame-ancestors 'self'** - This setting means that only pages from the same origin can embed the current web page using <frame>, <iframe>, <object>, or <embed>.
- **object-src 'none'** - This setting means that the use of <object> and <embed> elements is disabled.
- **worker-src 'self'** - This setting means that web workers can only be created from the same origin.

It is recommended that you do not change these settings. Changing these may increase the vulnerability of the AccessMatrix server or affect the functionality of certain AccessMatrix components.

However, it is possible to do so if necessary. Navigate to <ACMATRIX>/tomcat/webapps/am5/WEB-INF/, open **web.xml**, and search for the Content-Security-Policy (CSP) Header section to find the content security policy filter and its corresponding settings. Modify the value of the options parameter as needed.

3.2. Validate Tomcat Hardening

This section contains a checklist for validating that the bundled Tomcat has been hardened according to Center for Internet Security (CIS) guidelines.

i **Note:** In this section, <ACMATRIX_ROOT> refers to the AccessMatrix env variable or the full path of its root directory where AccessMatrix has been installed. For example: *C:\Program Files (x86)\AccessMatrix*

Remove Extraneous Resources

Remove Extraneous Files and Directories

Perform the following steps to validate that all unnecessary files and directories have been removed:

Windows	Execute the following command to validate that extraneous resources have been removed: <pre>dir %ACMATRIX_ROOT%\tomcat\webapps\examples %ACMATRIX_ROOT%\tomcat\webapps\docs %ACMATRIX_ROOT%\tomcat\webapps\ROOT %ACMATRIX_ROOT%\tomcat\webapps\host-manager %ACMATRIX_ROOT%\tomcat\webapps\manager</pre> i Note: If "File Not Found" is returned, there are no extraneous resources present.
Linux	Execute the following command to validate that extraneous resources have been removed: <pre>ls -l \$CATALINA_HOME\webapps\examples \$CATALINA_HOME\webapps\docs \$CATALINA_HOME\webapps\ROOT \$CATALINA_HOME\webapps\host-manager \$CATALINA_HOME\webapps\manager</pre> i Note: If "File Not Found" is returned, there are no extraneous resources present.

Disable Unused Connectors

Perform the following steps to disable all unused connectors:

Windows	1. Execute the following command to find all configured connectors: <code>findstr "Connector" %ACMATRIX_ROOT%\tomcat\conf\server.xml</code> 2. Navigate to <ACMATRIX_ROOT>/tomcat/conf/ and open the server.xml file. 3. Ensure that the required connectors are present and have not been commented out. 4. Comment out or remove any connector that is not required.
Linux	1. Execute the following command to find all configured connectors: <code>grep "Connector" \$CATALINA_HOME\conf\server.xml</code> 2. Navigate to <ACMATRIX_ROOT>/tomcat/conf/ and open the server.xml file.

	3. Ensure that the required connectors are present and have not been commented out. 4. Comment out or remove any connector that is not required.
--	---

Device Setup

Mask Server Infrastructure

By hiding details about the server infrastructure, it makes it more difficult for attackers to assess potential vulnerabilities specific to the server. This step includes validating several items:

- Alter the advertised server.info string
- Alter the advertised server.number string
- Alter the advertised server.built date

Perform the following:

Windows	1. Execute the following command to navigate to the directory: <code>cd %ACMATRIX_ROOT%/tomcat/lib</code> 2. Execute the following command to display the server details: <code>jar xf catalina.jar org/apache/catalina/util/ServerInfo.properties.</code> 3. Ensure that the server.info, server.number and server.built values have been changed from the default values.  Note: These values should not indicate the type of server, the version number, or the date it was compiled.
Linux	1. Execute the following command to navigate to the directory: <code>cd \$CATALINA_HOME/lib</code> 2. Execute the following command to display the server details: <code>jar xf catalina.jar org/apache/catalina/util/ServerInfo.properties.</code> 3. Ensure that the server.info, server.number and server.built values have been changed from the default values.  Note: These values should not indicate the type of server, the version number, or the date it was compiled.

Disable X-Powered-By HTTP Header and Rename the Server Value for All Connectors

Perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the **server.xml** file.
2. Locate the valid connector elements.

-
3. Set a **server** attribute for each valid connector.

Important Note: The server attribute must not reveal the underlying infrastructure. For example, "Apache Tomcat" or "AMServer" are not appropriate values.

4. Ensure that the **xpoweredBy** attribute for each valid connector is either not present, or has been set to "false".

Note: If left unspecified, the default value for this attribute is "false".

Disable client facing Stack Traces

Perform the following to determine if Tomcat has been configured not to send debug information to requestors:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the **web.xml** file.
2. Ensure that the **web-app** element contains the **error-page** child element.
3. Ensure that the **error-page** element contains the following child elements:
 - o **exception-type** with the value "java.lang.Throwable".
 - o **location**

XML

```
<error-page>
  <exception-type>java.lang.Throwable</exception-type>
  <location>/errorMessage.html</location>
</error-page>
```

4. Repeat the steps above for each application hosted within Tomcat.

Note: To locate the **web.xml** file of each application instance, navigate to `<ACMATRIX_ROOT>/tomcat/webapps/example_app/WEB-INF/`, where "example_app" is replaced with the name of the folder that represents the appropriate web application. For example, "am5".

Turn Off TRACE

Perform the following to determine if the server platform, as advertised in the HTTP server header has been changed:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the **server.xml** file.
-

-
2. Ensure that the **allowTrace** attribute for each valid connector is either not present, or has been set to "false".
 3. Navigate to <ACMATRIX_ROOT>/tomcat/webapps/example_app/WEB-INF/ and open the **web.xml** file.

i **Note:** This file path is an illustration. Replace "example_app" with the name of the folder that represents the appropriate web application. For example, "am5".

4. Add the following as a child of the **web-app** root element:

XML

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>restricted methods</web-
resource-name>
    ...
    <http-method>TRACE</http-method>
    ...
  </web-resource-collection>
  ...
</security-constraint>
```

5. Repeat steps 3 to 4 for each web application.

i **Note:** Each folder within <ACMATRIX_ROOT>/tomcat/webapps/ represents the respective web applications.

Ensure Server Header is Modified to Prevent Information Disclosure

In AccessMatrix, server headers have been removed from the **server.xml** file. As such, it is not necessary to validate this requirement.

i **Note:** Ensure that you have completed the steps in [Disable X-Powered-By HTTP Header and Rename the Server Value for all Connectors](#).

Protect Shutdown Port

Set a Nondeterministic Shutdown Command Value

Ensure that the shutdown attribute is not set to "SHUTDOWN". Perform the following:

1. Navigate to <ACMATRIX_ROOT>/tomcat/conf/ and open the **server.xml** file.
-

2. Set a nondeterministic value in the **shutdown** attribute. This attribute is located within the server element.

XML	
<code><Server port="8005" shutdown="U0hVVERPV05OT1c="></code>	
	Note: "U0hVVERPV05OT1c=" is the Base64 encoded equivalent of "SHUTDOWNNOW".

3. Save the changes.

To verify that the shutdown attribute has been configured correctly, perform the following:

1. Open Putty and access the desired AccessMatrix server.
2. Execute the command: `telnet localhost 8005`.
3. Execute the command: `U0hVVERPV05OT1c=`. The AccessMatrix server should shut down.

Disable the Shutdown Port

Ensure that the shutdown port has been disabled. Perform the following:

Windows	1. Navigate to <code><ACMATRIX_ROOT>/tomcat/conf/</code> and open the server.xml file. 2. Modify the Server port value to "-1": XML <code><Server port="-1" shutdown="SHUTDOWN"></code> 3. Save the changes. 4. Execute the following command to validate that the shutdown port has been disabled: <code>findstr /R "<Server[][^>]*port[]*=[]*\\"-1\\" server.xml</code>  Note: If configured correctly, there will not be any output. However, if "port 8005" is returned, the shutdown port has not been disabled and it is still using the default value.
Linux	1. Navigate to <code><ACMATRIX_ROOT>/tomcat/conf/</code> and open the server.xml file. 2. Modify the Server port value to "-1": XML <code><Server port="-1" shutdown="SHUTDOWN"></code> 3. Save the changes.

	4. Execute the following command to validate that the shutdown port has been disabled: <pre>grep '<Server[:space:]+\[^>*port[:space:]*=[[:space:]]*"-1"' server.xml</pre>
	i Note: If configured correctly, there will not be any output. However, if "port 8005" is returned, the shutdown port has not been disabled and it is still using the default value.

Protect Tomcat Configurations

i	Note: The following configurations must be done using a Linux environment.
----------	---

Restrict Access to \$CATALINA_HOME

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin.tomcat $CATALINA_HOME`
2. Execute the following command to modify the permissions: `chmod g-w,o-rwx $CATALINA_HOME`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME`
4. Execute the following command to validate that access to \$CATALINA_HOME has been restricted: `find . -follow -maxdepth 0 \( -perm /o+rx,g=w -o ! -user tomcat_admin -o ! -group tomcat \) -ls`

i	Note: If access has been restricted correctly, there will not be any output that is returned.
----------	--

Restrict Access to \$CATALINA_BASE

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin.tomcat $CATALINA_BASE`
2. Execute the following command to modify the permissions: `chmod g-w,o-rwx $CATALINA_BASE`
3. Enter the following command to navigate to the directory: `cd $CATALINA_BASE`
4. Execute the following command to validate that access to \$CATALINA_BASE has been restricted: `find . -follow -maxdepth 0 \( -perm /o+rx,g=w -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat Configuration Directory

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf`
2. Execute the following command to modify the permissions: `chmod g-w,o-rwx $CATALINA_HOME/conf`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf`
4. Execute the following command to validate that access to Tomcat configuration directory has been restricted: `find . -maxdepth 0 \( -perm /o+rx,g=w -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat Logs Directory

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/logs`
2. Execute the following command to modify the permissions: `chmod o-rwx $CATALINA_HOME/logs`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME`
4. Execute the following command to validate that access to Tomcat logs directory has been restricted: `find logs -follow -maxdepth 0 \( -perm /o+rx -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat Temp Directory

Perform the following steps:

-
1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/temp`
 2. Execute the following command to modify the permissions: `chmod o-rwx $CATALINA_HOME/temp`
 3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME`
 4. Execute the command to validate that access to Tomcat temp directory has been restricted: `find temp -follow -maxdepth 0 \( -perm /o+rx -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat Binaries Directory

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/bin`
2. Execute the following command to modify the permissions: `chmod g-w,o-rwx $CATALINA_HOME/bin`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME`
4. Execute the command to validate that access to Tomcat binaries directory has been restricted: `find bin -follow -maxdepth 0 \( -perm /o+rx,g=w -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat Web Application Directory

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/webapps`
 2. Execute the following command to modify the permissions: `chmod g-w,o-rwx $CATALINA_HOME/webapps`
 3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME`
-

-
4. Execute the command to validate that access to Tomcat web application directory has been

```
restricted: find webapps -follow -maxdepth 0 \( -perm /o+rw, g=w -o ! -user tomcat_admin -o ! -group tomcat \) -ls
```



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat catalina.properties

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/catalina.properties`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/catalina.properties`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
4. Execute the command to validate that access to Tomcat catalina.properties has been restricted: `find catalina.properties -follow -maxdepth 0 \( -perm /o+rw, g+rw, u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat catalina.policy

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/catalina.policy`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/catalina.policy`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
4. Execute the command to validate that access to Tomcat catalina.policy has been restricted: `find catalina.policy -follow -maxdepth 0 \( -perm /o+rw, g+rw, u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat context.xml

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/context.xml`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/context.xml`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf`
4. Execute the command to validate that access to Tomcat context.xml has been restricted: `find context.xml -follow -maxdepth 0 \( -perm /o+rw,g+rw,u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat logging.properties

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_BASE/conf/logging.properties`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_BASE/conf/logging.properties`
3. Execute the following command to navigate to the directory: `cd $CATALINA_BASE/conf/`
4. Execute the command to validate that access to Tomcat logging.properties has been restricted: `find logging.properties -follow -maxdepth 0 \( -perm /o+rw,g+rw,u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`



Note: If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat server.xml

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/server.xml`

-
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/server.xml`
 3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
 4. Execute the command to validate that access to Tomcat server.xml has been restricted: `find server.xml -follow -maxdepth 0 \( -perm /o+rw,g+rw,u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`

i **Note:** If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat tomcat-users.xml

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/tomcat-users.xml`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/tomcat-users.xml`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
4. Execute the command to validate that access to Tomcat tomcat-users.xml has been restricted: `find tomcat-users.xml -follow -maxdepth 0 \( -perm /o+rw,g+rw,u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`

i **Note:** If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to Tomcat web.xml

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/web.xml`
 2. Execute the following command to modify the permissions: `chmod 400 $CATALINA_HOME/conf/web.xml`
 3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
 4. Execute the command to validate that access to Tomcat web.xml has been restricted: `find web.xml -follow -maxdepth 0 \( -perm /o+rw,g+rw,u+wx -o ! -user tomcat_admin -o ! -group tomcat \) -ls`
-

i **Note:** If the access has been restricted correctly, there will not be any output that is returned.

Restrict Access to jaspic-providers.xml

Perform the following steps:

1. Execute the following command to set the necessary permissions: `chown tomcat_admin:tomcat $CATALINA_HOME/conf/jaspic-providers.xml`
2. Execute the following command to modify the permissions: `chmod 600 $CATALINA_HOME/conf/jaspic-providers.xml`
3. Execute the following command to navigate to the directory: `cd $CATALINA_HOME/conf/`
4. Execute the command to validate that access to Tomcat jaspic-providers.xml has been restricted: `find jaspic-providers.xml -follow -maxdepth 0 \( -perm /o+rw,wx,u+x -o ! -user tomcat_admin -o ! -group tomcat \) -ls`

i **Note:** If the access has been restricted correctly, there will not be any output that is returned.

Configure Realms

Use Secure Realms

To validate that unsecured realms are not in use, execute the following commands:

i **Note:** If configured correctly, each of the commands will not return any output.

Windows	1. <code>className" %ACMATRIX_ROOT%/tomcat/conf/server.xml findstr MemoryRealm findstr "Realm</code> 2. <code>className" %ACMATRIX_ROOT%/tomcat/conf/server.xml findstr JDBCRealm findstr "Realm</code> 3. <code>className" %ACMATRIX_ROOT%/tomcat/conf/server.xml findstr UserDatabaseRealm findstr "Realm</code> 4. <code>className" %ACMATRIX_ROOT%/tomcat/conf/server.xml findstr JAASRealm</code>
Linux	1. <code>className" \$CATALINA_HOME/conf/server.xml grep MemoryRealm</code> 2. <code>className" \$CATALINA_HOME/conf/server.xml grep JDBCRealm</code> 3. <code>className" \$CATALINA_HOME/conf/server.xml grep UserDatabaseRealm</code> 4. <code>className" \$CATALINA_HOME/conf/server.xml grep JAASRealm</code>

Use LockOut Realms

Locking user accounts after multiple failed login attempts helps to prevent attackers from brute forcing logins. In AccessMatrix, this can be achieved by configuring internal login policy. These configurations cater for a variety of login requirements and support security audits. As such, the use of **LockOutRealms** at the server level is not required.

Connector Security

To harden connector security, the connectors must first be configured using the **server.xml** file. These security requirements include:

- Ensure Client-cert authentication has been set up
- Ensure SSLEnabled is set to true for sensitive connectors
- Ensure scheme is set accurately
- Ensure secure is set to true only for SSL-enabled connectors
- Ensure 'sslProtocol' is configured correctly for secure connectors

Perform the following steps:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **server.xml** file.
2. Ensure that the **clientAuth** attribute has been set to "true" for all valid HTTPS connector configurations.
3. Ensure that the **certificateVerification** attribute has been set to "required" for all valid HTTPS connector configurations.
4. Ensure that the **SSLEnabled** attribute has been set to "true" for all valid connectors that will send or receive sensitive information.



Note: By default, the SSLEnabled attribute is set to "false".

5. Ensure that the **scheme** attribute is set to "https" for all valid connectors operating over HTTPS.



Note: By default, the scheme attribute is set to "http".

6. Ensure that the **secure** attribute is set to "true" for all valid connectors that will send or receive sensitive information.



Note: By default, the secure attribute is set to "false".

-
7. Ensure that the **sslProtocol** attribute is set to "TLS" for all valid connectors that will send or receive sensitive information.

An example of a connector that meets all of the above configurations is provided below:

XML

```
<Connector port="8443" minSpareThreads="10"
  protocol="com.isprint.amutil.tomcat.AMCustomNIO2Protocol"
  enableLookups="false" disableUploadTimeout="true"
  keepAliveTimeout="900000" maxKeepAliveRequests="-1"
  acceptCount="100" maxThreads="200"
  scheme="https" secure="true" SSLEnabled="true"
  maxParameterCount="1000"
>
  <SSLHostConfig certificateVerification="required" sslProtocol="TLS"
protocols="TLSv1.2,TLSv1.3"
ciphers="TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_AES_128
_GCM_SHA256,TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA
_WITH_AES_128_GCM_SHA256,TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,TLS_ECDH
E_ECDSA_WITH_CHACHA20_POLY1305_SHA256,TLS_ECDHE_RSA_WITH_CHACHA20_POLY130
5_SHA256,TLS_CHACHA20_POLY1305_SHA256"
truststoreFile="conf/testtrust.keystore"
truststorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1UsK2LzK8LJUyF2RQgyTe
2RU=" truststoreType="JKS">
    <Certificate certificateKeystoreFile="conf/testserver.keystore"
certificateKeystorePassword="AesEncryptor:txNbulL6Ip9JtvEFzUZ1UsK2LzK8LJU
yF2RQgyTe2RU=" certificateKeystoreType="JKS"/>
  </SSLHostConfig>
</Connector>
```

Establish and Protect Logging Facilities

Application Specific Logging

Specify File Handler in logging.properties Files

Application specific logging is recommended as part of CIS guidelines. In AccessMatrix, application specific logs will be generated. The **amlog4j2.properties** and **amlog4j2.xml** files contain the file handlers that are necessary for generating the application logs. These files contain file handlers that are similar to the default file handlers in **logging.properties**.

To see the list of handlers that have been specified, execute the following command: `grep handlers $CATALINA_BASE/webapps/am5/WEB-INF/classes/amlog4j2.properties`.

Ensure className is Set Correctly in context.xml

AccessMatrix Universal Access Management (UAM) uses Apache AccessLogValve at the server level with Tomcat. This is not covered within the CIS Apache Tomcat 10 Benchmark, as it only covers application level configurations. As this setup does not have any implications on access control, this validation step is not necessary.

Ensure Directory in context.xml is A Secure Location

It is not necessary to validate this requirement, as the AccessMatrix application folder permissions will have been secured in [Protect Tomcat Configurations](#). As such, the **context.xml** file located within the AccessMatrix application folder will also be secure.

Ensure Pattern in context.xml is Correct

UAM uses Log4j2 for logging instead of Apache AccessLogValve. Additionally, AccessMatrix does not use the */META-INF* directory that contains the **context.xml** file. As such, no changes are necessary for the **server.xml** file as it already meets the minimum requirements set by CIS.

Ensure Directory in logging.properties is A Secure Location

It is not necessary to validate this requirement, as the AccessMatrix application folder permissions will have been secured in [Protect Tomcat Configurations](#). As such, the **logging.properties** file located within the AccessMatrix application folder will also be secure.

Configure Catalina Policy

Restrict Runtime Access to Sensitive Package

Perform the following steps:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **catalina.properties** file.
2. Search for the **package.access** attribute.
3. Ensure that only allowed packages have been defined within the **package.access** attribute. For example:

Properties (.properties files)

```
package.access=sun.,org.apache.catalina.,org.apache.coyote.,org.apache.jasper.,org.apache.tomcat.
```

Application Deployment

Disable Auto Deployment of Applications

To validate that applications will not be automatically deployed, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the **server.xml** file.
2. Ensure that the **autoDeploy** attribute is present, and has been set to "false".

Disable Application Startup on Deployment

By default, applications will not automatically deploy during startup, as the attribute **deployOnStartup** is not enabled. Ensure that this attribute in the **server.xml** file is either not present, or has been set to "false".

Miscellaneous Configuration Settings

Ensure Web Content Directory Is on Separate Partition

The web content directory must be on a separate partition from the Tomcat system files. Perform the following steps:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/webapps` and move all contents to a different partition. For example, `<ACMATRIX_ROOT2>/webapps`.
2. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the **server.xml** file.
3. Set the value of the **appBase** attribute to the file path of the new partition you have created in the first step. For example:

XML

```
appBase = "<ACMATRIX_ROOT2>/webapps"
```

Restrict Access to Web Administration Application

RemoteAddressValve functions to whitelist IP addresses, which is similar to access control at the Tomcat server level. By default, AccessMatrix does not configure RemoteAddressValve. As such, no validation is required for this requirement. However, it can still be configured using the **server.xml** file.



Note: When configuring RemoteAddressValve, ensure that the regular expression is set to allow or deny a range of IP addresses.

Configurations for Manager Application

The Tomcat Manager Application is not included within AccessMatrix and the associated folders have been removed.



Note: For more details, refer to [Remove Extraneous Resources Files and Directories](#).

As such, the following validations for the Tomcat Manager Application will not be required.

- Restrict Manager Application
- Force SSL when accessing Manager Application
- Rename Manager Application
- Ensure Manager Application passwords are encrypted

Enable Strict Servlet Compliance

By default, Accessmatrix has been configured to enable strict servlet compliance. To validate that these requirements have been met, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/bin` and open the **setenv.bat** file using a code editor.
2. Ensure that the following can be found within the file:

Code

```
set -  
Dorg.apache.catalina.STRICT_SERVLET_COMPLIANCE=true
```

Disable Session Facade Recycling

By default, Accessmatrix has been configured to disable session facade recycling. To validate that these requirements have been met, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/bin` and open the **setenv.bat** file using a code editor.
2. Ensure that the following can be found within the file:

Code

```
set -
Dorg.apache.catalina.connector.RECYCLE_FACADES=true
```

Do Not Allow Additional Path Delimiters

By default, Accessmatrix has been configured to not allow additional path delimiters. To validate that these requirements have been met, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/bin` and open the **setenv.bat** file using a code editor.
2. Ensure that the following can be found within the file:

Code

```
set -
Dorg.apache.catalina.connector.CoyoteAdapter.ALLOW_BACKSLASH=false
-Dorg.apache.tomcat.util.buf.UDecoder.ALLOW_ENCODED_SLASH=false
```

Configure connectionTimeout

Ensure that the connection timeout has been configured for the server. To configure this attribute, perform the following steps:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **server.xml** file.
2. Modify the value for the connectionTimeout attribute.

XML

```
connectionTimeout = "60"
```

Configure maxHttpHeaderSize

By default, this attribute has been set to 8192 bytes by Tomcat.

i **Note:** This attribute can be configured using the **server.xml** file found in `<ACMATRIX_ROOT>/tomcat/conf/`.

Force SSL For All Applications

There are no actions required if the connectors have been configured correctly.

i **Note:** For more details, refer to [Connector Security](#).

Disable Symbolic Linking

In Tomcat, symbolic links are disabled by default. To validate this, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **context.xml** file.
2. Ensure that the **allowLinking** attribute is either not present, or has been set to "false".

i **Note:** If left unspecified, the default value for this attribute is "false".

Disable Running Applications as Privileged

In Tomcat, applications do not run as privileged by default. To validate this, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **context.xml** file.
2. Ensure that the **privileged** attribute is either not present, or has been set to "false".

i **Note:** If left unspecified, the default value for this attribute is "false".

Disable Cross Context Request

No actions are required as AccessMatrix is deployed as a standalone application. Without cross context requests, malicious applications are not able to make requests to restricted applications.

Do Not Resolve Hosts on Logging Valves

No further hardening actions are required as the **enableLookup** attribute is set to "false" by default.

To validate this, perform the following:

-
1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf` and open the **server.xml** file.
 2. Ensure that **enableLookup** attribute has been set to "false" within each valid **Connector** element. For example:

XML

```
<Connector port="8443" minSpareThreads="10"
protocol="com.isprint.amutil.tomcat.AMCustomNIO2Protocol"
enableLookups="false"
disableUploadTimeout="true"
keepAliveTimeout="900000"
maxKeepAliveRequests="-1"
acceptCount="100" maxThreads="200"
scheme="https" secure="true"
SSLEnabled="true"
maxParameterCount="1000"
>
```

Enable Memory Leak Listener

No actions are required as this is enabled by default in UAM.

Set Security Lifecycle Listener

The Security Lifecycle Listener can enforce a blacklist of OS users that must not be used to start the Tomcat server. If there are no specific users to blacklist, this setting is not applicable.

Use logEffectiveWebXml and metadata-complete

These settings must be enabled for deploying applications in production. By default, AccessMatrix has been configured to meet this requirement. To validate, perform the following:

1. Navigate to `<ACMATRIX_ROOT>/tomcat/conf/` and open the web.xml file.
2. Ensure that the attribute **metadata-complete** within the **web-app** element has been set to "true".

XML

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
version="6.0" metadata-complete="true">
```

-
- Repeat step 2 for each web application.



Note: To locate the **web.xml** file of each application instance, navigate to `<ACMATRIX_ROOT>/tomcat/webapps/example_app/WEB-INF/`, where "example_app" is replaced with the name of the folder that represents the appropriate web application. For example, "am5".

- Navigate to `<ACMATRIX_ROOT>/tomcat/conf/Catalina/localhost/` and open the **am5.xml** file.
- Ensure that the attribute **logEffectiveWebXml** within the **Context** element has been set to "true".

XML

```
<Context logEffectiveWebXml="true">
```

4. AccessMatrix Server Hardening

The following sections contain information on hardening the AccessMatrix server:

- [API Access](#)
- [Key Management](#)

4.1. API Access

SSL with Client Authentication

AccessMatrix Server APIs uses client certificate as part of SSL connection to authenticate the client applications connecting to AccessMatrix Server. This needs to be done by connecting to web application server's HTTPS port that requires client authentication or allow client authentication. When a client is creating a connection to this port, client authentication will be done as part of the SSL handshake process. Client certificate will be presented to the web application server. Client certificate or its issuer must be trusted by the web application server as per normal certificate validation. AccessMatrix Server API will then validate the certificate's identity and link it (by DN) to a non-interactive user (e.g. usoagent) in AccessMatrix Server. Depending on the APIs, the API call will then use the identity of this non-interactive user to validate its access rights.

Thus, it is imperative that the **application makes use of a proper client certificate** and the **non-interactive user, linked to the certificate, has the intended access rights**. Some modules in AccessMatrix has an access agent restriction configuration to allow certain module to be used by a certain non-interactive user.

Disable Simulated Certificate

As mentioned in earlier section, client applications need to use client certificates to authenticate their identities when calling AccessMatrix Server APIs. By default, AccessMatrix server is configured with simulated certificate to allow ease of testing. This simulated certificate allows non-authenticated connection (such as non-SSL HTTP port) to use the simulated certificate as the client certificate. This must be disabled in production environment.

1. Open **amsystem.properties**.
2. Remove or comment out this property:

Properties (.properties files)

```
com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file
```

3. Save the file. Restart is required for the changes to take effect.
-

Dedicated client certificate for each application group/cluster

Since AccessMatrix Server uses the client certificate and non-interactive user to identify the application, depending on the requirement, different application group/cluster may benefit from the use of different certificate/non-interactive user so that access control can be further differentiated and audit information can also be more meaningful.

4.2. Key Management

HSM-based Protection

AccessMatrix key management has a flexible hierarchy that allows keys used to encrypt data to be wrapped/protected by an HSM key. For example, Attribute Encryption Key (AEK), Token Encryption Key (TEK), PIN Encryption Key (PEK) can all be protected by an HSM key. This means when the keys are stored in database, they are encrypted by key that resides in HSM. Thus, in the event of database is compromised, the keys used to protect the data are secured by a separate system (HSM).

The steps below outline how to create the encryption keys. You may refer to the [Common Administration Guide - Key Management](#) section for detailed instructions on this.

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key**.
2. Create an encryption key for key wrapping. This key needs to be created using the HSM-based key provider.
 - a. Under the **Attributes** header, ensure the value of the **Purpose** attribute is set to "Key Wrapping".
3. Create an encryption key that will be used to encrypt the data.
 - a. Under the **Attributes** header, ensure the value of the **Purpose** is set to the value that reflects its the intended purpose (e.g. "Attribute Encryption", "PIN Encryption", etc.).
 - b. Under the **Storage and Key Wrapping** header, ensure that:
 - i. The value of **Encoding Format** is set to "Wrapped by other key"
 - ii. The value of **Encoder wrapper key id** is set to the wrapper key created in previous step.
4. Navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**.
5. Configure the System Key Provider to use the new encryption key based on its purpose as the default key. The Encryption key for some modules may also need to be configured within the module configuration.

Using HSM key directly for data encryption

It is technically possible to use HSM key directly to encrypt data directly. However, note that HSM comes with different performance levels. Thus, it is generally safer to use the HSM key as a wrapping key only. However, for certain modules, such as Pseudo E2EE Authentication, Token seed encryption, where the data are considered very sensitive since they are authentication related, HSM key can be used directly to encrypt these data.

Securing the HSM connection/credential

In order to use HSM, AccessMatrix Server needs to make a connection to an HSM. This connection needs to be secured. How they are secured differ based on the HSM model. Normally these are the possible ways:

- Client IP whitelisting
- Client-authenticated SSL
- Password/PIN
- Or combination of the above

Most of the time, the connection to the HSM is handled by HSM's client driver/utility, not by AccessMatrix Server. However, when there is a need to configure the HSM password/PIN in AccessMatrix Server, these need to be specified in **amsystem.properties**.

AccessMatrix provides encryptor utility (**encryptor.bat** or **encryptor.sh** in <ACMATRIX_HOME>/bin/) that can be used to provide encryption of the HSM password/PIN so that it does not appear in plaintext in the amsystem.properties. The encryption itself requires a secret to be provided in a separate file. Access to the file needs to be secured using file permission of the OS (Linux: 600, Windows: allow only user used to run AM that may access the file). This encryption helps to avoid accidental exposure when **amsystem.properties** needs to be provided to other party (e.g. i-Sprint support) for support purpose.

Refer to [Common Deployment Guide - Encryptor Utility](#) section. Ensure to choose AesEncryptor or Aes256Encryptor and to provide the user salt (secret) using file.
